

2026-08-31 Update: λ Re-tuned on the Training Set + ChainRAG/GRITHopper Reproduction Comparison

Two things this week: 1) instead of tuning λ directly on the validation set (dev), re-sweep it on a training-set subset to avoid the risk of "the same data being used for tuning and for the final reported evaluation"; 2) reproduce ChainRAG and GRITHopper, two prior works close to this project's own approach, and compare retrieval recall and final-answer quality against D-group.

I. λ Re-tuned on the Training Set

1.1 Motivation

The existing gate v3 D-group's $\lambda=0.50$ was swept directly on the validation set (dev) — this is only a one-dimensional scalar search with limited risk, but the cleaner practice is to select the hyperparameter on data the model has never been tuned against, then confirm on dev, rather than letting "tuning" and "the evaluation set used in the final report" fully coincide. This round switches to sweeping λ on a **training-set subset**, with dev used only for the final confirmation.

1.2 Method

- From MuSiQue train (GT decompose), draw a K-stratified sample of 3000 examples (1000 each for K=2/3/4), and generate the corresponding **BART decompose** for them (not GT — the BART decomposer itself was trained on train, so testing directly with GT, or testing BART-on-train naively, would be distorted; it was specifically confirmed that BART's output on this 3000-example train subset still carries genuine generation noise and is not "memorized" down to GT-quality clean text — see the conversation record for details).
- Sweep $\lambda \in [0.10, 0.80]$ in steps of 0.05 on these 3000 examples, using the current full method (rawprefix + comparison_hint reader prompt).

1.3 Results: the λ Curve on the Train Subset

λ	recall@1	chain@1	EM	F1
0.10	0.7252	0.4710	0.4193	0.5005
0.20	0.7515	0.5133	0.4413	0.5271
0.30	0.7555	0.5183	0.4527	0.5414
0.40	0.7561	0.5217	0.4543	0.5426
0.50	0.7558	0.5223	0.4540	0.5425
0.60	0.7556	0.5250	0.4593	0.5460
0.65	0.7540	0.5220	0.4587	0.5461
0.70	0.7523	0.5210	0.4580	0.5453
0.80	0.7500	0.5163	0.4567	0.5437

EM/F1 peak at 0.60-0.65, and all four metrics drop together past 0.70 — a genuine internal optimum region, and higher than the 0.50 originally tuned on dev.

1.4 Full-Dev Validation: 0.50 $\rightarrow$ 0.60

Ran $\lambda=0.60$ once on the full dev set of all three datasets as a direct validation check:

Dataset	Metrics	$\lambda=0.50$	$\lambda=0.60$	Δ
MuSiQue	recall@1/chain@1/EM/F1	0.7018/0.5143/0.4427/0.5386	0.6994/0.5118/0.4386/0.5341	all slightly down (EM -0.41pp)

Dataset	Metrics	$\lambda=0.50$	$\lambda=0.60$	Δ
2WikiMultihopQA	same	0.9289/0.8613/0.5982/0.6770	0.9316/0.8685/0.6023/0.6819	all slightly up (EM +0.41pp)
HotpotQA	same	0.6595/0.5567/0.5517/0.6870	0.6620/0.5610/0.5519/0.6883	all slightly up

Mixed results, but the direction matters: $\lambda=0.50$ was originally tuned mainly against MuSiQue, and 0.60 happens to dip slightly on MuSiQue while improving on both of the other two datasets that had zero part in the tuning — indicating 0.60 is a more robust cross-dataset value, not a number overfit to a single dataset. $\lambda=0.60$ is adopted as the new default (the `--lambda-gate` default in `run_retrieval_exp_wavefront_gate_v3_rawprefix.py` has been updated accordingly).

1.5 Final Comparison: No Gate (Baseline) vs. New $\lambda=0.60$ (Full Method)

Dataset	Setup	recall@1	recall@3	chain@1	EM	F1
MuSiQue	baseline (no gate)	0.6003	0.7611	0.3335	0.3757	0.4652
	+ gate ($\lambda=0.60$, full method)	0.6994	0.8435	0.5118	0.4386	0.5341
2WikiMultihopQA	baseline (no gate)	0.8123	0.9021	0.6047	0.4544	0.5230
	+ gate ($\lambda=0.60$, full method)	0.9316	0.9842	0.8685	0.6023	0.6819
HotpotQA	baseline (no gate)	0.5746	0.7760	0.3361	0.4918	0.6163
	+ gate ($\lambda=0.60$, full method)	0.6620	0.8701	0.5610	0.5519	0.6883

Adding the gate improves every metric on all three datasets clearly: MuSiQue EM +6.3pp/F1 +6.9pp, 2Wiki EM +14.8pp/F1 +15.9pp, HotpotQA EM +6.0pp/F1 +7.2pp — the gate's fused signal (+ rawprefix + the reader-prompt fix + this round's re-tuned λ) still holds a solid gain over the pure cosine-retrieval baseline.

II. ChainRAG / GRITHopper Reproduction Comparison

Cloned and localized two open-source works whose approach is close to this project's own — ChainRAG (ACL 2025) and GRITHopper (EACL 2026) — and re-ran them on this project's own three datasets (MuSiQue/2WikiMultihopQA/HotpotQA, distractor setting, not each work's own native evaluation setup), under a unified metric definition, for a three-way comparison against D-group. Directories: `chainrag/`, `grithopper/` (both at the workspace root, independent git repos, not affecting this repo). All three datasets, all three methods, fully run.

2.1 ChainRAG's Algorithm, with a Real Example

ChainRAG ("Mitigating Lost-in-Retrieval Problems in Retrieval Augmented Multi-Hop QA")'s core idea: first build the whole context into a "sentence graph"; when retrieving for each sub-question, don't settle for the first-pass ranking — if it's not enough, expand along entity connectivity in the graph, until "enough" or the budget runs out. Four steps:

- 1. Build the sentence graph (once per question):** split every candidate passage of this question into sentences, extract the entities in each sentence, and connect any two sentences that share an entity with an edge. This graph is shared by every sub-question afterward.
- 2. Decide whether/how to decompose (LLM call):** first ask once whether this question needs multi-step reasoning; if so, decompose it into sub-questions.
- 3. Process each sub-question in turn (the core loop):**
 - Seed retrieval:** run the sub-question through an embedding model, compute cosine similarity against every sentence in this question's pool, take the top-100 (under our distractor setting a question usually has only 60-100 sentences total, so this step barely filters anything — it's effectively a full rerank of the whole pool), then cross-encoder rerank down to the top-7 seeds.
 - Ask the LLM whether that's enough to answer:** if yes, generate this sub-question's answer directly and move to the next sub-question; if not, proceed to the next step.

- **Graph expansion:** pull in all 1-hop neighbors of the seed sentences in the sentence graph and ask again; if still not enough, expand to 2-hop, then 3-hop, until it's enough or a 3000-word budget is used up, at which point an answer is forced.
- 4. **Synthesize the final answer:** once every sub-question has been processed, feed the list of "sub-question + its own answer" together with the original question to the LLM to generate the final answer.

A real example (MuSiQue dev, question "Who led the military expedition in the province that borders Zhejiang to the south?", gold answer Chen Zheng, gold passages are *Zhejiang* and *Hokkien*):

ChainRAG decomposed this question into:

Sub-question 1: "Which province borders Zhejiang to the south?"
 Sub-question 2: "Who led the military expedition in that province?"

For sub-question 2, the #1 slot of **ranked_titles** is exactly *Hokkien* (the gold passage — the retrieval itself is correct). But the *Hokkien* article actually mentions "military expedition" twice (Chen Zheng's in 677 AD, suppressing a rebellion, and Wang Chao/Wang Shenzhi's in 885 AD, suppressing the aftermath of the Huang Chao rebellion), and after graph expansion the context accumulated for this question averages sentences from about 10 different titles (roughly half of this question's candidate pool) — descriptions of both events ended up in the final generation input. **ChainRAG's final answer was "Wang Chao and Wang Shenzhi"** — it picked the later-era event, judged wrong (EM=0). Retrieval hit the gold passage exactly, yet the final answer was still wrong; the root cause is that graph expansion's "cast a wide net" strategy stuffed too many competing candidate facts into the context, diluting the correct signal.

2.2 GRITHopper's Algorithm, on the Same Example

GRITHopper ("Decomposition-Free Multi-Hop Dense Retrieval")'s approach is the opposite: **no sub-question decomposition at all**. It uses a specially fine-tuned 7B model (GritLM-7B backbone + GritHopper fine-tuned weights, trained on MuSiQue, 2Wiki, HotpotQA, EX-FEVER, and HoVer), and retrieves iteratively straight from the original question plus whatever evidence has been retrieved so far:

1. **Encode the query:** concatenate "the original question + the evidence retrieved so far" into one prompt and encode it into a single vector with the model (no distinction is made for which hop this is — every hop sees "original question + accumulated evidence," with no sub-question separately generated for "what this particular hop should look for").
2. **Compute similarity, take top-1:** compute cosine similarity against every candidate passage vector for this question, take the highest-scoring one, and commit it as this hop's retrieval result.
3. **Accumulate evidence, repeat:** append the just-committed passage to the "evidence so far" list and go back to step 1, until the hop count is reached (we fix this at the gold hop count K, rather than using its own automatic-stopping mechanism, to avoid conflating "when to stop" with "how accurate the retrieval is" in the evaluation).

On this same question, GRITHopper hit gold on both hops (*Zhejiang* → *Hokkien*), an evidence chain just as precise as D-group's, and **its final answer correctly gave "Chen Zheng"** (EM=1) — though this is just one example among many; GRITHopper's overall answer accuracy is in fact noticeably low, explained in detail in section 2.5.

Also worth recording, a counterexample (revealing a structural weakness of the "decomposition-free" design): question "Where in Zhejiang is the city where Protestants are especially notable?", gold answer Yongjia County, gold passages are *Sanjiang Church* and *Zhejiang*. GRITHopper **retrieved the same Zhejiang article on both hops** (never actually obtaining the second passage, *Sanjiang Church*) — because every hop's query is only "original question + evidence so far," with no sub-question explicitly pointing to "what this hop should look for," the model failed to realize it needed to find a different, more specific article, and ended up "circling" on the same one. Its final answer was "Wenzhou" (the closest information it could extract from the only passage it retrieved, *Zhejiang*), while the true gold answer "Yongjia County" is in *Sanjiang Church*, which it never retrieved at all — judged wrong. This kind of "repeatedly hitting the same document without genuinely exploring new evidence" is part of why GRITHopper's recall@1 is high while its "full-coverage rate" is noticeably lower (see the numbers in section 2.4).

2.3 Strict Definition of the Unified Metrics

All three methods are aligned to the same set of definitions, not each method's own paper definitions:

- **recall@1 / recall@3 (order-invariant)**: for every retrieval decision unit (a "hop" for us, a "sub-question" for ChainRAG, "each step of iterative retrieval" for GRITHopper), take its own ranked candidate list; a hit is counted if the top-1 / top-3 contains **any one of** this question's gold passages (not necessarily the one assigned to that specific position). Hit decision units / total decision units = recall@k. "Order-invariant" was chosen because it is the only definition that can be applied to all three methods on equal footing: our own "hop" has a dataset-annotation-guaranteed positional correspondence to a specific gold passage, but neither ChainRAG's nor GRITHopper's own decomposition/iteration carries that same guarantee (self-decomposed or self-iterated, with no guarantee that hop count or order aligns with the dataset's own annotation). On the D-group side, `run_retrieval_exp_wavefront_gate_v3_rawprefix.py` was specifically extended with new `full_pool_gold_rank_orderinvariant_*` fields, computed live from the full candidate pool (before any pruning) under this exact definition — not stitched together after the fact.
- **Per-question gold coverage / full-coverage rate**: for each question, take the union of **all evidence actually committed/used across the whole trace** (for D-group and GRITHopper this is "the one final pick per hop," K hops → K passages; for ChainRAG it is "every sentence's source title that ended up in the context fed to answer generation for a given sub-question," which after graph expansion is often several dozen), intersect it with this question's gold passage set, and report the fraction of gold covered; coverage = 1.0 (no gold missed) counts as one full coverage. **This metric is mutually comparable between D-group and GRITHopper (both are "exactly one pick per hop, never more"), but neither is directly comparable to ChainRAG's coverage** — ChainRAG's candidate base is inherently several times larger (graph expansion folds in a large number of non-top-1 sentences too), so its higher coverage largely reflects "more candidates," not "more precise selection."
- **EM / F1**: the standard HotpotQA official convention (`normalize_answer` — strip articles, punctuation, case — plus token-overlap F1), shared across all three methods and directly comparable. **All three methods' final answer generation uses the exact same local Llama-3.1-8B-Instruct** (D-group already used it; ChainRAG's `LLM.py` and GRITHopper's `run_answer_gen.py` were both changed to call the same local vLLM server), so the comparison isolates differences in "retrieval + reasoning design," not differences in generator capability.

2.4 Full Three-Way Comparison (All Three Datasets, Full Scale)

Dataset	Method	recall@1	recall@3	gold coverage	full-coverage rate	EM	F1
MuSiQue	D-group ($\lambda=0.60$)	0.8552	0.9622	0.7733	0.5283	0.4377	0.5324
	ChainRAG	0.6716	0.8628	0.9587	0.9048	0.2764	0.3847
	GRITHopper	0.9058	0.9863	0.8207	0.6090	0.2793	0.3734
2WikiMultihopQA	D-group	0.9723	0.9970	0.9407	0.8799	0.6023	0.6819
	ChainRAG	0.9199	0.9785	0.9870	0.9696	0.4426	0.5220
	GRITHopper	0.9500	0.9964	0.8922	0.7751	0.5390	0.6267
HotpotQA	D-group	0.8988	0.9805	0.8532	0.7317	0.5519	0.6883
	ChainRAG	0.8032	0.9496	0.9864	0.9741	0.4598	0.6027
	GRITHopper	0.9658	0.9959	0.9464	0.9068	0.5546	0.6926

The conclusion is consistent across all three datasets: recall@1/@3 ranks GRITHopper > D-group

ChainRAG; gold coverage / full-coverage rate ranks ChainRAG > GRITHopper $\approx$ D-group (ChainRAG wins via its graph-expansion "cast a wide net" approach, but this metric's basis is not on equal footing between the two sides, see 2.3); **EM/F1 ranks D-group clearly ahead, with GRITHopper and ChainRAG trading wins/losses and both clearly behind D-group** — on 2Wiki/HotpotQA GRITHopper overtakes ChainRAG (its retrieval advantage converts into a partial answer advantage there), but on MuSiQue the two are nearly tied on EM (0.2793 vs. 0.2764), showing that how reliably a retrieval advantage converts into an answer advantage is inconsistent — the answer-synthesis stage (whether there's a decomposition scaffold, how clean the context is) is the key variable that actually determines the final result. Two patterns in the table stand out as the most counter-intuitive: **ChainRAG has the highest coverage of the three yet the lowest EM/F1; GRITHopper has the highest recall of the three yet its EM/F1 still clearly trails D-group.** These two "high score on one metric, doesn't translate to good answers" phenomena have different root causes, examined separately below.

2.5 Why ChainRAG and GRITHopper Underperform D-group — Each Method's Own Problem

2.5.1 ChainRAG's Problem: Graph Expansion's "Wide Net" Dilutes the Correct Signal

ChainRAG is the only one of the three methods that is entirely untrained (off-the-shelf embedding model + off-the-shelf cross-encoder, zero parameters adjusted for these three datasets), and its recall is also the lowest of the three — but its real problem isn't recall; it's that **even when retrieval hits gold, the answer can still be wrong**. The real example in section 2.1 already shows this: the #1 retrieval slot for sub-question 2 is exactly the gold passage *Hokkien*, but this article happens to describe "military expedition" twice (Chen Zheng's in 677 AD, Wang Chao/Wang Shenzhi's in 885 AD), and graph expansion — to guarantee "enough" — pulled in sentences from about 10 different titles on average for this question into the final generation context. Descriptions of both events ended up in the input together, and **the final answer picked the later-era event ("Wang Chao and Wang Shenzhi"), judged wrong**. Retrieval hit gold precisely, yet the answer was still wrong — the root cause is graph expansion's "grab more rather than make a choice" strategy, which drowns the correct signal in a pile of equally plausible-looking candidate facts. This is also exactly what explains ChainRAG's contradictory combination of the highest gold coverage of the three (its candidate base is inherently larger) yet the lowest EM/F1.

2.5.2 GRITHopper's Problem: Huge Retrieval Investment, but No Decomposition Scaffold — All the Answer-Synthesis Burden Is Pushed to the End

GRITHopper's problem is of a different nature: it is the **most accurate retriever** of the three methods, yet its answer quality is likewise clearly behind D-group — on MuSiQue, recall@1=0.9058 (highest of the three), EM=0.2793 (lowest of the three, 15.8 percentage points below D-group's 0.4377). Most accurate retrieval, worst answers — explaining this contrast takes two steps: first why its retrieval is so strong, then why that retrieval advantage fails to carry through to the answer.

Why the retrieval is strong — a comparison of the three methods' retrieval mechanisms and investment: the recall ranking (GRITHopper > D-group > ChainRAG) tracks investment level fairly directly:

	ChainRAG	D-group (ours)	GRITHopper
Retrieval model	Off-the-shelf general-purpose embedding model + off-the-shelf general-purpose cross-encoder reranker, zero training (used directly, not a single parameter trained on these three datasets)	BAAI/bge-base-en-v1.5 (110M params, general-purpose embedding model, zero training) + a specially trained small gate (PCA+LR / PCA+bilinear, parameter count in the thousands to tens of thousands, trained only on MuSiQue hidden states)	GritLM-7B backbone (7B params) + the entire model specially fine-tuned , training data covers five datasets: MuSiQue, 2Wiki, HotpotQA, EX-FEVER, HoVer
Has it seen these three datasets	Never (pure prompting, zero-shot)	The gate was trained on MuSiQue (GT-decompose hidden states)	All three datasets are in its training data
Structural design	bi-encoder coarse rank + cross-encoder rerank + entity-graph expansion (uses graph structure to compensate for imprecise retrieval, doesn't aim to pick right in one shot)	bi-encoder retrieval + small gate rerank (pick right in one shot, no expansion/fallback)	Dedicated model directly encodes "question + accumulated evidence," picks right in one shot (no expansion/fallback — the same design philosophy as D-group)
Training cost	None	Low (small gate, single-GPU, hours-scale)	High (7B model fine-tuning, needs a multi-dataset training pipeline; the paper's own training cost is far higher than our gate's)

GRITHopper is the only one of the three that has "fine-tuned a 7B-scale model specifically for the multi-hop retrieval task as a whole," with training data directly covering the three datasets used in this evaluation — fundamentally, its retrieval-precision advantage is bought with heavy investment, not some architectural insight that neither we nor ChainRAG

thought of. D-group sits in the middle because, although its backbone retriever (BGE) is just as untrained as ChainRAG's, it adds a specially trained small gate on top for correction — **a very small training investment buys a retrieval precision clearly above the purely-untrained approach (ChainRAG)**, which is exactly the reason this gate exists in the first place.

Why the retrieval advantage fails to carry through to the answer, illustrated with a real example of what our reader's input actually looks like (MuSiQue dev, question "Who led the military expedition in the province that borders Zhejiang to the south?"):

```
Reasoning trace:
Step 1:
Subquestion: What province borders Zhejiang to the south?
Selected evidence: "Zhejiang ... is bordered by ... Fujian province to the south ..."
Answer: Fujian

Step 2:
Subquestion: Who led the military expedition in Jiangxi
Selected evidence: "In 677 (during the reign of Emperor Gaozong), Chen Zheng (陳政),
together with his son Chen Yuanguang (陳元光), led a military expedition to pacify
the rebellion in Fujian. ... In 885, ... Wang Chao (王潮) and Wang Shenzhi (王審知),
led a military expedition force to pacify the Huang Chao rebellion. ..."
Answer: NA

Re-read the original question before answering:
Who led the military expedition in the province that borders Zhejiang to the south?

Answer the original question directly using only the reasoning trace above.
Give a short span or phrase only (entity, date, number, or yes/no).
If the original question asks you to COMPARE two or more entities ..., the per-step
answers above are the values used to make that comparison, not the final answer
themselves ...

Output exactly one line:
Final answer: <short answer>
```

There's a very telling detail in this real example: **hop 2's own short-answer extraction failed (Answer: NA)** — BART mis-wrote this hop's sub-question as "in Jiangxi" (it should have been Fujian; the BART decomposition itself has an error), which caused the short-answer extraction module to give up and output NA, since the question and evidence don't line up on the entity name. **Yet the final reader still correctly answered "Chen Zheng"** (EM=1) — because it was also given this hop's **raw evidence text in full** (not just the failed short answer), and after re-reading the original question it was able to find the answer directly from the raw text, without relying entirely on that failed intermediate step.

This is exactly the core advantage of our prompt format: redundant signals, a structured scaffold — every hop gives the reader three things at once (the sub-question, the raw evidence text in full, and this hop's short answer); even if one of them goes wrong (short-answer extraction fails, a sub-question is mis-phrased), the reader still has another signal to fall back on. And the "reasoning trace" structured format effectively pre-organizes, for the reader, "what each hop should be about, what was found on the previous hop" — leaving the reader with only the comparatively simple task of "synthesize the already-organized information and answer the original question."

GRITHopper's reader has none of this scaffolding — being decomposition-free, it never produces sub-questions or per-hop short answers at all; all it can give the reader is a flat "list of raw documents in retrieval order":

```
Original question: Who led the military expedition in the province that borders
Zhejiang to the south?

Retrieved documents (in retrieval order):
Document 1: Zhejiang. Zhejiang ... is bordered by ... Fujian province to the south ...
Document 2: Hokkien. In 677 ..., Chen Zheng ... led a military expedition to pacify
the rebellion in Fujian. ... In 885, ... Wang Chao and Wang Shenzhi, led a military
expedition force to pacify the Huang Chao rebellion. ...
```

Answer the original question.

Even when every step retrieved gold exactly (as in this question), the reader still has to **work out, entirely on its own, the whole "which passage answers which hop, and how these passages chain together" reasoning** — our pipeline has already done this work for it (decomposing sub-questions, extracting a short answer at each hop), GRITHopper does none of it, and pushes all of it onto this one final generation step. When faced with a passage like the *Hokkien* one above, which happens to describe two similar events (two "military expeditions" from different eras) in the same passage, a reader with no intermediate guidance is more likely to pick the wrong one — GRITHopper happened to get this particular question right, but its overall answer accuracy statistics (section 2.4) are noticeably low, showing this kind of failure is not an isolated case.

Is it "because there's no decomposition" — yes, and this is GRITHopper's own core design choice (being "Decomposition-Free" is literally its selling point): not depending on a decomposer buys the benefit of not worrying about decomposition quality dragging retrieval down, and generalizing more robustly to out-of-distribution data (this is also part of why its retrieval score is high, see the investment comparison above); but the cost is that the "decompose + answer step by step" capability is removed entirely from the retrieval stage and pushed onto the final generation step to make up for — the gains won on the retrieval side get eaten away to a greater extent on the generation side. ChainRAG does decompose into sub-questions and does generate per-hop answers, but because graph expansion stuffs the context with too much noisy information (section 2.5.1), it likewise fails to realize the potential of that decomposition step. **Our method is the only one of the three that gets all three things right at once: precise retrieval + structured decomposition + redundant multi-signal context** — this is most likely the root reason for its across-the-board EM/F1 lead, not just a lead on the retrieval-score dimension alone.

In one sentence: the three methods represent three different trade-offs — ChainRAG (zero training, graph-expansion fallback, coarse answer synthesis), GRITHopper (heavy training bought for retrieval precision, no decomposition pushing the entire answer-synthesis burden to the end), D-group (a lightly-trained gate for correction + a complete decomposition-and-redundant- signal reader) — and the latter, with far less retrieval investment than GRITHopper, achieves the best final-answer quality on all three datasets by making sure retrieval is "good enough" while keeping the reasoning structure complete.